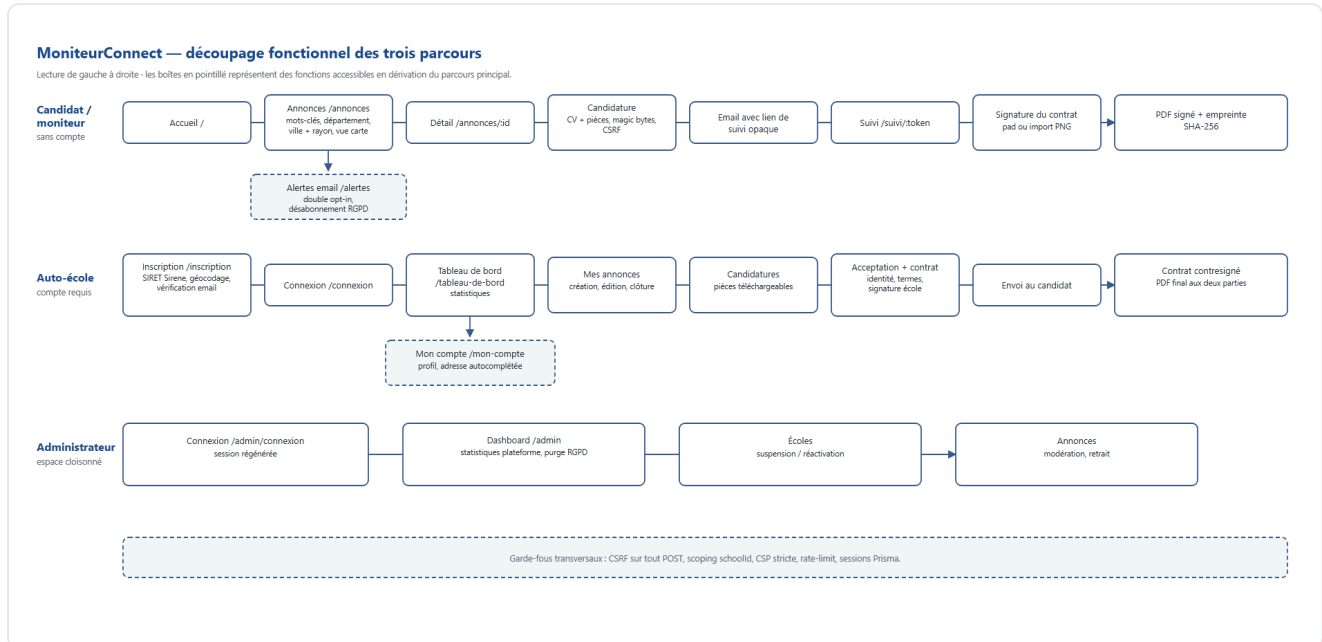


Découpage fonctionnel — les trois parcours

Date : 2026-07-11. Vue fonctionnelle destinée au jury : qui fait quoi, dans quel ordre, et où chaque étape vit dans le code. Le découpage technique par lots (A → L) reste documenté dans `resume-projet.md` et `AGENTS.md`.



Source vectorielle : [diagrammes/parcours-fonctionnels.svg](#).

1. Parcours candidat / moniteur — sans compte

Le moniteur ne crée jamais de compte : tout passe par un jeton de suivi opaque reçu par email.

1. L'accueil `/` est servi par `src/controllers/pageController.js` et ouvre le parcours public.
2. La liste `/annonces`, pilotée par `src/controllers/listingController.js`, combine mots-clés, département, ville et rayon avec une vue carte.
3. Le détail `/annonces/:id`, dans le même contrôleur, présente l'annonce choisie.
4. La candidature est reçue par `src/controllers/applicationController.js` avec le CV et les pièces, après contrôles des magic bytes et du jeton CSRF.
5. Un email transmet alors au candidat son lien de suivi opaque.
6. La page `/suivi/:token`, gérée par `src/controllers/trackingController.js`, expose l'avancement sans compte ni donnée personnelle dans l'URL.
7. `src/controllers/signatureController.js` permet ensuite de signer le contrat avec le pad ou l'import d'une image PNG.
8. Le parcours aboutit au PDF signé accompagné de son empreinte SHA-256.

En dérivation de la recherche, `src/controllers/alertController.js` gère les alertes email sous `/alertes`, avec double opt-in et désabonnement RGPD.

2. Parcours auto-école — compte requis

1. `src/controllers/authController.js` prend en charge l'inscription `/inscription`, la vérification SIRET Sirene, le géocodage et la vérification de l'email.
2. Le même contrôleur authentifie l'école sur `/connexion` et ouvre sa session.
3. `src/controllers/dashboardController.js` alimente `/tableau-de-bord` avec les statistiques de l'école.
4. `src/controllers/listingController.js`, monté sous `/mes-annonces` par `src/routes/manageRoutes.js`, couvre la création, l'édition et la clôture des annonces.
5. `src/controllers/applicationController.js` présente les candidatures reçues et leurs pièces téléchargeables.
6. `src/controllers/contractController.js` conduit l'acceptation, la saisie de l'identité et des termes, puis la signature de l'école.
7. Le même contrôleur envoie le contrat au candidat.
8. Après contresignature, le PDF final est mis à disposition des deux parties.

En dérivation, `src/controllers/accountController.js` gère `/mon-compte`, le profil de l'école et son adresse autocomplétée.

3. Parcours administrateur — cloisonné

1. `src/controllers/adminAuthController.js` gère `/admin/connexion`; la connexion régénère la session et ferme l'espace école.
2. `src/controllers/adminController.js` alimente le dashboard `/admin`, ses statistiques plateforme et la purge RGPD.
3. Le même contrôleur permet de suspendre ou de réactiver les écoles.
4. Il assure enfin la modération et le retrait des annonces.

Garde-fous transversaux

- CSRF sur tous les POST (y compris multipart); CSP stricte sans inline.
- Cloisonnement: `requireAuth`, `requireAdmin`, scoping `schoolId` (une école reçoit 404 sur les documents d'une autre).
- Uploads: taille, mimetype et magic bytes vérifiés; stockage hors public/.
- RGPD: purge automatique quotidienne + purge manuelle admin; désabonnement des alertes en un clic.

Chaque étape est couverte par la suite de tests (15 fichiers, 448 assertions) — la correspondance critère → test est dans `expression-du-besoin-v2.md`.